CLIMATE & AIR QUALITY MONITORING

SIC Coding and Programming Project

Submitted by

JASPREET KAUR [RA2311030010321]
ANANYA PARIYANI [RA2311003011340]
E KRISHNA SAI REVANTH [RA2311030010340]

JUNE 2026

PROBLEM STATEMENT

The objective of this project is to build a **Climate and Air Quality Monitoring System** that processes hourly pollution data from multiple Indian cities, cleans and transforms the dataset, calculates Air Quality Index values, classifies pollution severity, detects critical pollution periods, and presents the results through visual dashboards and maps.

The system focuses on the following major goals:

- Clean raw sensor data by handling missing timestamps, invalid pollutant values, inconsistent city names, and sensor spikes.
- Compute AQI using pollutant readings such as PM25, PM10, NO2, SO2, CO, and O3.
- Categorize AQI values into health-risk levels such as Good, Moderate, Unhealthy, Very Unhealthy, and Hazardous.
- Identify rush-hour pollution patterns and detect cities with consecutive critical AQI conditions.
- Use data structures such as Stack and Priority Queue to manage alert detection and high-risk AQI prioritization.
- Generate analytical visualizations including AQI trends, heatmaps, pollutant comparison charts, category breakdowns, and interactive risk maps.
- Export processed results such as daily city-wise AQI summaries for further reporting and analysis.

SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB, recommended 8 GB
- Storage: At least 500 MB free disk space
- Display: Standard monitor with 1366 × 768 resolution or higher
- Internet: Required only if installing Python packages or viewing online map resources

Software Requirements

- Operating System: Windows, Linux, or macOS
- Programming Language: Python 3.8 or above
- Code Editor: Visual Studio Code, PyCharm, Jupyter Notebook, or any Python-supported IDE
- Browser: Google Chrome, Microsoft Edge, or Firefox for viewing HTML map/dashboard files

Python Libraries Required

- `pandas` for data cleaning, grouping, and CSV processing
- `numpy` for numerical operations and random data generation
- `matplotlib` for dashboard visualization
- `seaborn` or Matplotlib seaborn style for improved chart appearance
- `folium` for generating interactive AQI risk maps
- `branca` for map color scales and legends

Input Requirements

- Raw climate and air-quality dataset containing fields such as:
  - `timestamp`
  - `city`
  - `PM25`
  - `PM10`
  - `NO2`
  - `SO2`
  - `CO`

- 03

Output Requirements

- Cleaned dataset with corrected missing and invalid values
- Computed AQI values
- AQI category labels
- Rush-hour flag
- Daily average AQI CSV file: `daily_aqi.csv`
- Dashboard image: `climate_dashboard.png`
- Interactive AQI map: `aqi_risk_map.html`

Functional Requirements

- The system should remove rows with missing timestamps.
- The system should clean pollutant values such as PM10 unit suffixes.
- The system should replace invalid sensor readings like `-9999`.
- The system should cap extreme pollutant spikes.
- The system should fill missing pollutant values using city-wise medians.
- The system should compute AQI from pollutant values.
- The system should classify AQI into health-risk categories.
- The system should detect rush-hour pollution records.
- The system should identify cities with consecutive critical AQI readings.
- The system should generate visual reports and export processed results.

TASK 1

AQI Data Pipeline (Pandas + ETL)

Task 1 focuses on building a complete data cleaning and transformation pipeline for air-quality data using Python and Pandas. The raw dataset contains hourly pollution readings from multiple Indian cities, but the data includes missing timestamps, invalid sensor values, inconsistent city names, unit suffixes, and extreme pollutant spikes.

The main objective of this task is to convert the raw, noisy dataset into a clean and structured dataset that can be used for AQI calculation, analysis, visualization, and reporting.

The pipeline performs the following operations:

- Removes rows where the timestamp is missing.
- Cleans the `PM10` column by removing the `µg/m³` unit suffix and converting values to numeric format.
- Replaces invalid `CO` sensor values such as `-9999` with `NaN`.
- Caps extreme `PM25` spike values using the 99th percentile.
- Converts city names into proper Title Case format.
- Fills missing values in `PM25`, `CO`, `SO2`, and `NO2` using city-wise median values.
- Computes AQI using pollutant values:
  `PM25`, `PM10`, `NO2/2`, `SO2`, `CO*10`, and `O3/2`.
- Assigns AQI categories such as `Good`, `Moderate`, `Unhealthy`, `Very Unhealthy`, and `Hazardous`.
- Adds a `Rush_Hour` flag based on morning and evening traffic hours.
- Extracts useful time-based features such as hour, day of week, and date.
- Generates daily city-wise average AQI values.
- Exports the final processed result as `daily_aqi.csv`.

This task forms the foundation of the project because all later analysis, alert detection, dashboards, and maps depend on the cleaned and processed AQI dataset created in this pipeline.

Task 1 Code:

```
#Task 1: AQI Data Pipeline

print("\n")
print("TASK 1 — AQI DATA PIPELINE (ETL)")
print("\n")

# Subtask 1: Drop NaT timestamps
before = len(df)
df = df.dropna(subset=['timestamp'])
print(f"\n[1] Dropped {before - len(df)} NaT timestamp rows → {len(df)} rows remain")

# Subtask 2: Clean PM10 — strip µg/m³ suffix, cast to float
df['PM10'] = df['PM10'].astype(str).str.replace(r'\s*µg/m³', '', regex=True)
df['PM10'] = pd.to_numeric(df['PM10'], errors='coerce')
print(f"[2] PM10 string suffixes stripped and cast to float")

# Subtask 3: Replace CO -9999 sentinel with NaN; cap PM25 spikes to 99th pct
df['CO'] = pd.to_numeric(df['CO'], errors='coerce')
df['CO'] = df['CO'].replace(-9999, np.nan)
df['PM25'] = pd.to_numeric(df['PM25'], errors='coerce')
p99 = df.loc[df['PM25'] < 9999, 'PM25'].quantile(0.99)
spike_count = (df['PM25'] >= 9999).sum()
df['PM25'] = df['PM25'].clip(upper=p99)
print(f"[3] CO sentinel -9999 → NaN | {spike_count} PM25 spikes capped at 99th pct ({p99:.1f})")

# Subtask 4: Normalise city to Title Case; fill NaN with city-wise medians
df['city'] = df['city'].str.title()
for col in ['PM25', 'CO', 'SO2', 'NO2']:
    df[col] = df.groupby('city')[col].transform(lambda x: x.fillna(x.median()))
print(f"[4] City names -> Title Case | NaN in PM25/CO/SO2/NO2 filled with city medians")

# Subtask 5: Compute AQI, add AQI_Category & Rush_Hour, export CSV
```

```
df['timestamp'] = pd.to_datetime(df['timestamp'])
df['AQI'] = df.apply(
    lambda r: max(r['PM25'], r['PM10'], r['NO2']/2, r['SO2'], r['CO']*10, r['O3']/2),
    axis=1
).round(1)

def aqi_category(v):
    if v < 50:  return 'Good'
    if v < 100: return 'Moderate'
    if v < 200: return 'Unhealthy'
    if v < 300: return 'Very Unhealthy'
    return 'Hazardous'

df['AQI_Category'] = df['AQI'].apply(aqi_category)
df['Rush_Hour'] = (
    df['timestamp'].dt.hour.between(7, 9) |
    df['timestamp'].dt.hour.between(17, 19)
)
df['hour']        = df['timestamp'].dt.hour
df['day_of_week'] = df['timestamp'].dt.dayofweek
df['date']        = df['timestamp'].dt.date

daily_aqi = df.groupby(['date', 'city'])['AQI'].mean().round(1).reset_index()
daily_aqi.columns = ['date', 'city', 'avg_AQI']
daily_aqi.to_csv('daily_aqi.csv', index=False)
print(f"[5] AQI computed | AQI_Category & Rush_Hour added | daily_aqi.csv saved
({len(daily_aqi)} rows)")
print(f"\n    AQI Category Distribution:\n{df['AQI_Category'].value_counts().to_string()}")
```

TASK 2

Pollution Alert System — Stack + Priority Queue (DSA)

Task 2 focuses on building a pollution alert system using basic data structures implemented from scratch. After the AQI dataset is cleaned and processed in Task 1, this task uses the calculated AQI values to identify high-risk pollution situations and prioritize alerts.

The main objective of this task is to apply **Data Structures and Algorithms** concepts to air-quality monitoring. A `Stack` is used to detect consecutive critical AQI readings, while a `PriorityQueue` is used to rank pollution alerts based on AQI severity.

The task includes the following operations:

- Implement a `Stack` data structure from scratch using a Python list.
- Implement a `PriorityQueue` from scratch using a max-heap.
- Use the `Stack` to track consecutive hours where AQI is greater than `200`.
- Detect cities that experience 3 or more consecutive critical AQI hours.
- Use the `PriorityQueue` to store AQI alerts with AQI value as priority.
- Dequeue alerts in descending order of AQI, so the most severe pollution cases are handled first.
- Display ranked alerts showing city, timestamp, and AQI value.
- Print the list of cities affected by continuous critical pollution periods.

This task demonstrates how DSA concepts can be used in a real-world environmental monitoring system. The stack helps identify continuous danger periods, while the priority queue ensures that the highest-risk AQI alerts are reported first.

Task 2 Code:

```
#Task 2: Pollution Alert System — Stack + Priority Queue (DSA)


print("\n")
print("TASK 2 — POLLUTION ALERT SYSTEM (DSA)")
print("\n")

# Subtask 1: Stack and PriorityQueue from scratch (no external libs)
class Stack:
    def __init__(self):
        self._data = []

    def push(self, item):
        self._data.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack underflow")
        return self._data.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self._data[-1]

    def is_empty(self):
        return len(self._data) == 0

    def __len__(self):
        return len(self._data)
```

```
class PriorityQueue:
    """Highest AQI dequeued first."""
    def __init__(self):
        self._heap = []

    def enqueue(self, item, priority):
        self._heap.append((priority, item))
        self._sift_up(len(self._heap) - 1)

    def dequeue(self):
        if not self._heap:
            raise IndexError("PriorityQueue is empty")
        self._swap(0, len(self._heap) - 1)
        _, item = self._heap.pop()
        if self._heap:
            self._sift_down(0)
        return item

    def _sift_up(self, i):
        while i > 0:
            p = (i - 1) // 2
            if self._heap[i][0] > self._heap[p][0]:
                self._swap(i, p)
                i = p
            else:
                break

    def _sift_down(self, i):
        n = len(self._heap)
        while True:
            largest = i
```

```
            for child in [2*i+1, 2*i+2]:
                if child < n and self._heap[child][0] > self._heap[largest][0]:
                    largest = child
            if largest == i:
                break
            self._swap(i, largest)
            i = largest

    def _swap(self, i, j):
        self._heap[i], self._heap[j] = self._heap[j], self._heap[i]

    def is_empty(self):
        return len(self._heap) == 0

    def __len__(self):
        return len(self._heap)


# Subtask 2: Push AQI > 200 alerts onto Stack
alert_stack = Stack()
for _, row in df[df['AQI'] > 200].iterrows():
    alert_stack.push({'city': row['city'], 'timestamp': row['timestamp'], 'AQI': row['AQI']})
print(f"[2] {len(alert_stack)} alerts (AQI > 200) pushed onto Stack")

# Subtask 3: Transfer all Stack alerts to PriorityQueue using AQI as priority (higher = more urgent).
pq = PriorityQueue()
while not alert_stack.is_empty():
    alert = alert_stack.pop()
    pq.enqueue(alert, priority=alert['AQI'])
print(f"[3] All {len(pq)} Stack alerts transferred to PriorityQueue (priority = AQI)")
```

```
# Subtask 4: Dequeue and print Top 10 most critical
print("\n[4] Top 10 Most Critical Pollution Alerts :")
print(f"  {'Rank':<5} {'City':<13} {'Timestamp':<22} {'AQI':>8}")
print("  " + "-" * 52)
for rank in range(1, 11):
    if pq.is_empty():
        break
    a = pq.dequeue()
    print(f"  {rank:<5} {a['city']:<13} {str(a['timestamp']):<22} {a['AQI']:>8.1f}")

# Subtask 5: Detect 3-hour consecutive critical windows per city
print("\n[5] 3-Hour Consecutive Critical Windows (AQI > 200):")
critical_cities = []
for c in df['city'].unique():
    city_df = df[df['city'] == c].sort_values('timestamp')
    win = Stack()
    for _, row in city_df.iterrows():
        if row['AQI'] > 200:
            win.push(row['AQI'])
            if len(win) >= 3:
                critical_cities.append(c)
                break
        else:
            win = Stack()

critical_cities = sorted(set(critical_cities))
print(f"  Cities with 3+ consecutive critical hours: {critical_cities}")
print(f"  Total affected: {len(critical_cities)} cities")
```

TASK 3

Climate Dashboard (Matplotlib)

Task 3 focuses on creating a visual dashboard for analyzing climate and air-quality patterns using Matplotlib. After the AQI data is cleaned in Task 1 and alerts are detected in Task 2, this task presents the processed data in a clear graphical format.

The main objective of this task is to build a multi-panel dashboard that helps users understand AQI trends, pollution intensity, pollutant behavior, and city-wise air-quality distribution.

The dashboard includes the following visual components:

- A daily average AQI time-series chart for all cities.
- A critical AQI threshold line at `AQI = 200`.
- A shaded critical zone to highlight dangerous AQI levels.
- A heatmap showing average AQI by hour of day and day of week.
- A grouped bar chart comparing average pollutant levels across AQI categories.
- Normalized pollutant values so different pollutants can be compared on the same scale.
- City-wise pie charts showing AQI category distribution for each city.
- A shared AQI category legend using consistent colors.
- A dashboard title and organized 2×2 layout using Matplotlib GridSpec.

This task helps convert processed AQI data into meaningful visual insights. The dashboard allows users to quickly identify high-pollution days, risky hours, pollutant patterns, and city-level air-quality conditions. It also supports better understanding of pollution trends across Indian cities during January 2024.

Task 3 Code:

```
# Task 3:  Climate Dashboard (Matplotlib)



print("\n")

print("TASK 3 — CLIMATE DASHBOARD (MATPLOTLIB)")

print("\n")



plt.style.use('seaborn-v0_8-whitegrid')

CAT_COLORS = {

    'Good': '#2ecc71', 'Moderate': '#f1c40f',

    'Unhealthy': '#e67e22', 'Very Unhealthy': '#e74c3c', 'Hazardous': '#8e44ad'

}



fig = plt.figure(figsize=(22, 20), facecolor='#f8f9fa')

fig.suptitle('India Air Quality Monitoring Dashboard — January 2024',

             fontsize=18, fontweight='bold', color='#2c3e50', y=0.98)

outer = gridspec.GridSpec(2, 2, figure=fig, hspace=0.38, wspace=0.32,

                          left=0.06, right=0.97, top=0.94, bottom=0.04)



# Panel 1: 24h avg AQI time series

ax1 = fig.add_subplot(outer[0, 0])

city_palette = plt.cm.tab10(np.linspace(0, 1, len(cities)))
```

```
daily_city = df.groupby(['date', 'city'])['AQI'].mean().reset_index()

daily_city['date'] = pd.to_datetime(daily_city['date'])

aqi_max = daily_city['AQI'].max()

ax1.axhspan(200, aqi_max * 1.05, color='#e74c3c', alpha=0.12, label='Critical Zone (AQI > 200)')

ax1.axhline(200, color='#e74c3c', linewidth=1.2, linestyle='--', alpha=0.7)

for i, c in enumerate(cities):

    cdf = daily_city[daily_city['city'] == c].sort_values('date')

    ax1.plot(cdf['date'], cdf['AQI'], label=c, color=city_palette[i], linewidth=1.4, alpha=0.85)

ax1.set_title('Daily Avg AQI — All Cities', fontweight='bold', fontsize=11, pad=8)

ax1.set_xlabel('Date', fontsize=9)

ax1.set_ylabel('AQI', fontsize=9)

ax1.legend(ncol=2, fontsize=7.5, loc='upper right', framealpha=0.85)

ax1.tick_params(axis='x', rotation=30, labelsize=8)

ax1.tick_params(axis='y', labelsize=8)



# Panel 2: Heatmap hour_of_day vs day_of_week

ax2 = fig.add_subplot(outer[0, 1])

hm = df.groupby(['hour', 'day_of_week'])['AQI'].mean().unstack(fill_value=0)

hm = hm.reindex(index=range(24), columns=range(7), fill_value=0)

im = ax2.imshow(hm.values, aspect='auto', cmap='RdYlGn_r', interpolation='nearest')

cb = plt.colorbar(im, ax=ax2, pad=0.02, shrink=0.9)

cb.set_label('Avg AQI', fontsize=9)
```

```
cb.ax.tick_params(labelsize=8)

ax2.set_xticks(range(7))

ax2.set_xticklabels(['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'], fontsize=8)

ax2.set_yticks(range(0, 24, 3))

ax2.set_yticklabels([f'{h:02d}:00' for h in range(0, 24, 3)], fontsize=7.5)

ax2.set_title('AQI Heatmap: Hour of Day vs Day of Week', fontweight='bold', fontsize=11, pad=8)

ax2.set_xlabel('Day of Week', fontsize=9)

ax2.set_ylabel('Hour of Day', fontsize=9)



# Panel 3: Grouped bar — avg pollutant per AQI category (normalised 0-1)

ax3 = fig.add_subplot(outer[1, 0])

df['CO_x10'] = df['CO'] * 10

cat_order = ['Good', 'Moderate', 'Unhealthy', 'Very Unhealthy', 'Hazardous']

present_cats = [c for c in cat_order if c in df['AQI_Category'].unique()]

cat_poll = df.groupby('AQI_Category')[['PM25', 'PM10', 'NO2', 'SO2', 'CO_x10', 'O3']].mean()

cat_poll = cat_poll.reindex(present_cats)

for col in cat_poll.columns:

    mn, mx = cat_poll[col].min(), cat_poll[col].max()

    cat_poll[col] = (cat_poll[col] - mn) / (mx - mn + 1e-9)

x = np.arange(len(present_cats))

width = 0.13

bar_colors = ['#3498db', '#2ecc71', '#e67e22', '#e74c3c', '#9b59b6', '#1abc9c']
```

```
poll_labels = ['PM25', 'PM10', 'NO2', 'SO2', 'CO×10', 'O3']

for j, (poll, col, lbl) in enumerate(zip(['PM25', 'PM10', 'NO2', 'SO2', 'CO_x10', 'O3'], bar_colors, poll_labels)):

    ax3.bar(x + j * width, cat_poll[poll].values, width, label=lbl, color=col, alpha=0.85)

ax3.set_xticks(x + width * 2.5)

ax3.set_xticklabels(present_cats, rotation=15, fontsize=8.5)

ax3.set_ylim(0, 1.15)

ax3.set_ylabel('Normalised Value (0–1)', fontsize=9)

ax3.set_title('Avg Pollutant Levels by AQI Category (Normalised)', fontweight='bold', fontsize=11, pad=8)

ax3.legend(fontsize=8, ncol=3, loc='upper left')

ax3.tick_params(labelsize=8)



# Panel 4: 2×5 pie charts per city

inner = gridspec.GridSpecFromSubplotSpec(2, 5, subplot_spec=outer[1, 1], hspace=0.55, wspace=0.35)

all_cats = ['Good', 'Moderate', 'Unhealthy', 'Very Unhealthy', 'Hazardous']

pie_colors = [CAT_COLORS[c] for c in all_cats]

for k, c in enumerate(cities):

    row_idx, col_idx = divmod(k, 5)

    ax = fig.add_subplot(inner[row_idx, col_idx])

    counts = df[df['city'] == c]['AQI_Category'].value_counts()

    vals = [counts.get(cat, 0) for cat in all_cats]

    non_zero = [(v, pc) for v, pc in zip(vals, pie_colors) if v > 0]
```

```
    if non_zero:
        v_nz, c_nz = zip(*non_zero)
        ax.pie(v_nz, colors=c_nz, startangle=90, wedgeprops=dict(linewidth=0.5, edgecolor='white'))
    ax.set_title(c, fontsize=7.5, fontweight='bold', pad=3)


legend_handles = [Patch(color=CAT_COLORS[cat], label=cat) for cat in all_cats]
fig.legend(handles=legend_handles, loc='lower right', ncol=1, fontsize=8,
           title='AQI Categories', title_fontsize=8.5,
           bbox_to_anchor=(0.99, 0.04), framealpha=0.9)
ax_t = fig.add_axes([0.51, 0.475, 0.46, 0.02])
ax_t.axis('off')
ax_t.text(0.5, 0.5, 'AQI Category Breakdown per City',
          ha='center', va='center', fontsize=11, fontweight='bold', color='#2c3e50')


plt.savefig('climate_dashboard.png', dpi=200, bbox_inches='tight', facecolor='#f8f9fa')
plt.close()
print("Dashboard Created")
```

TASK 4

AQI Risk Map (Folium)

Task 4 focuses on creating an interactive AQI risk map using Folium. After cleaning the AQI data and generating dashboard visualizations, this task displays city-wise air-quality risk geographically on an India map.

The main objective of this task is to represent AQI severity using map markers, colors, and popups so users can quickly identify high-risk cities and compare pollution conditions across locations.

The AQI risk map includes the following features:

- An interactive map centered on India.
- City-wise AQI markers using latitude and longitude coordinates.
- Marker colors based on AQI category such as `Good`, `Moderate`, `Unhealthy`, `Very Unhealthy`, and `Hazardous`.
- Circle markers sized or styled according to pollution severity.
- Popups showing city name, average AQI, AQI category, and related pollution details.
- A legend explaining AQI category colors.
- Export of the final interactive map as `aqi_risk_map.html`.

This task helps transform processed air-quality data into a geographical visualization. The Folium map makes it easier to identify regional pollution patterns, compare AQI levels across Indian cities, and understand which locations are experiencing higher environmental risk.

Task 4 Code:

```
#  Task 4: AQI Risk Map (Folium)



print("\n")

print("TASK 4 — AQI RISK MAP (FOLIUM)")

print("\n")



# Subtask 1: Init map

m = folium.Map(location=[20.5937, 78.9629], zoom_start=5, tiles='CartoDB dark_matter')

print("\n[1] Folium map initialised — CartoDB dark_matter, zoom 5")



def get_worst_pollutant(r):

    return max(

        [('PM25', r['PM25']), ('PM10', r['PM10']),

         ('NO2', r['NO2']/2), ('SO2', r['SO2']),

         ('CO', r['CO']*10), ('O3', r['O3']/2)],

        key=lambda x: x[1]

    )[0]
```

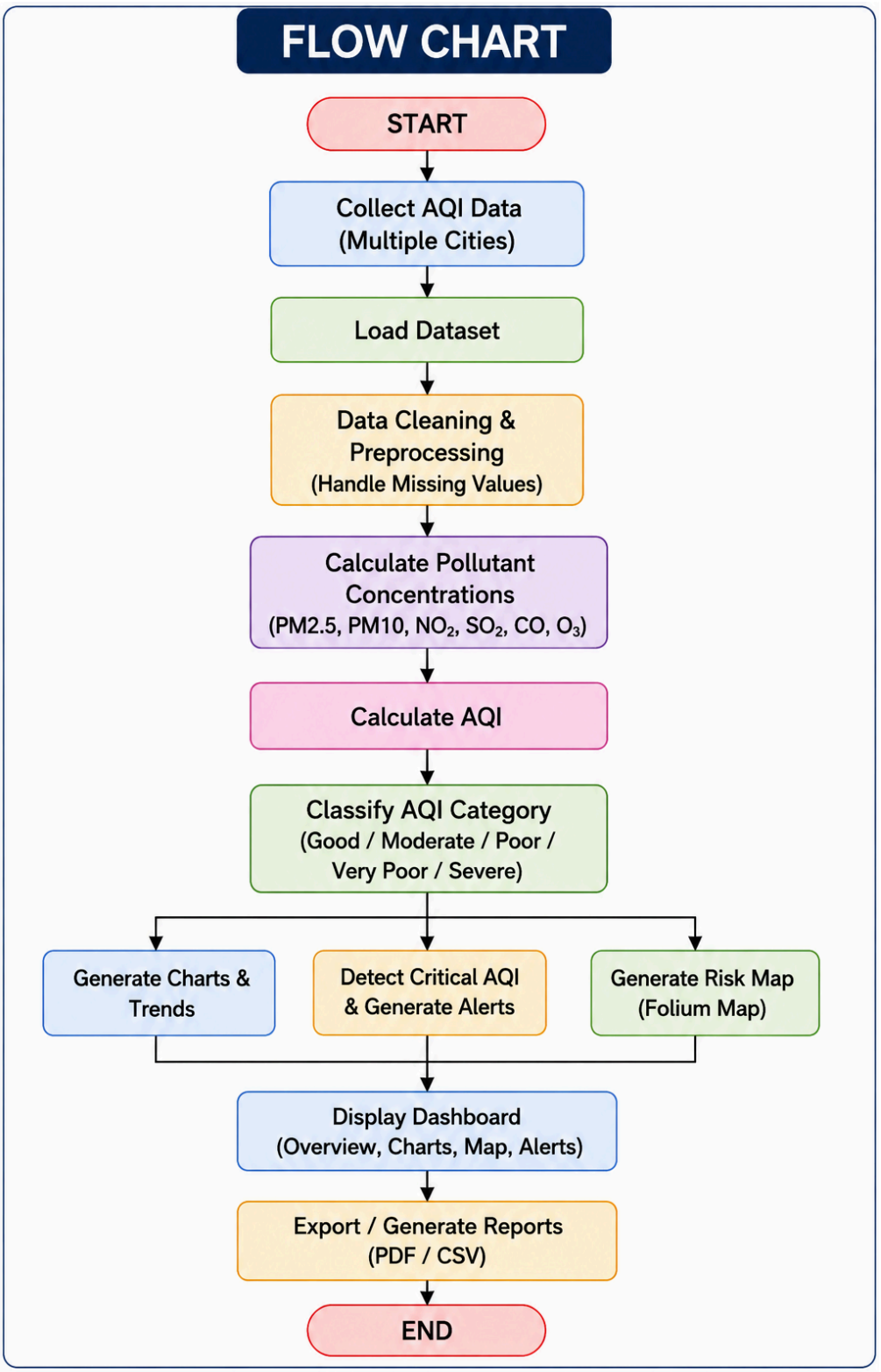
```
df['worst_pollutant'] = df.apply(get_worst_pollutant, axis=1)

city_stats = df.groupby('city').agg(
    avg_AQI=('AQI', 'mean'),
    rush_AQI=('AQI', lambda x: x[df.loc[x.index, 'Rush_Hour']].mean()),
    offpeak_AQI=('AQI', lambda x: x[~df.loc[x.index, 'Rush_Hour']].mean()),
    top_category=('AQI_Category', lambda x: x.mode()[0]),
    worst_poll=('worst_pollutant', lambda x: x.mode()[0])
).reset_index()
city_stats['aqi_delta'] = (city_stats['rush_AQI'] - city_stats['offpeak_AQI']).round(1)


def aqi_color(v):
    if v < 50:  return 'green'
    if v < 100: return 'yellow'
    if v < 200: return 'orange'
    if v < 300: return 'red'
    return 'darkred'


# Subtasks 2 + 3 + 5: CircleMarkers with popups in FeatureGroups
all_cats = ['Good', 'Moderate', 'Unhealthy', 'Very Unhealthy', 'Hazardous']
cat_groups = {cat: folium.FeatureGroup(name=f'AQI: {cat}', show=True) for cat in all_cats}


for _, row in city_stats.iterrows():
```

```
    c = row['city']
    lat, lon = CITY_COORDS[c]
    color = aqi_color(row['avg_AQI'])
    radius = max(6, row['avg_AQI'] / 10)
    delta_color = '#c0392b' if row['aqi_delta'] > 0 else '#27ae60'

    popup_html = f"""
    <div style="font-family:Arial,sans-serif;min-width:210px;padding:10px;">
                            <h3 style="margin:0 0 8px;color:#2c3e50;border-bottom:2px solid
#3498db;padding-bottom:4px;">{c}</h3>
      <table style="width:100%;font-size:13px;border-collapse:collapse;">
        <tr><td style="padding:3px 6px;color:#666;">Avg AQI</td>
            <td style="padding:3px 6px;font-weight:bold;color:{color};">{row['avg_AQI']:.1f}</td></tr>
        <tr style="background:#f8f9fa;"><td style="padding:3px 6px;color:#666;">Category</td>
            <td style="padding:3px 6px;font-weight:bold;">{row['top_category']}</td></tr>
        <tr><td style="padding:3px 6px;color:#666;">Worst Pollutant</td>
            <td style="padding:3px 6px;font-weight:bold;color:#e74c3c;">{row['worst_poll']}</td></tr>
                <tr style="background:#f8f9fa;"><td style="padding:3px 6px;color:#666;">Rush Hour
AQI</td>
            <td style="padding:3px 6px;">{row['rush_AQI']:.1f}</td></tr>
        <tr><td style="padding:3px 6px;color:#666;">Off-Peak AQI</td>
            <td style="padding:3px 6px;">{row['offpeak_AQI']:.1f}</td></tr>
        <tr style="background:#fff3cd;"><td style="padding:3px 6px;color:#666;">Rush Δ</td>
```

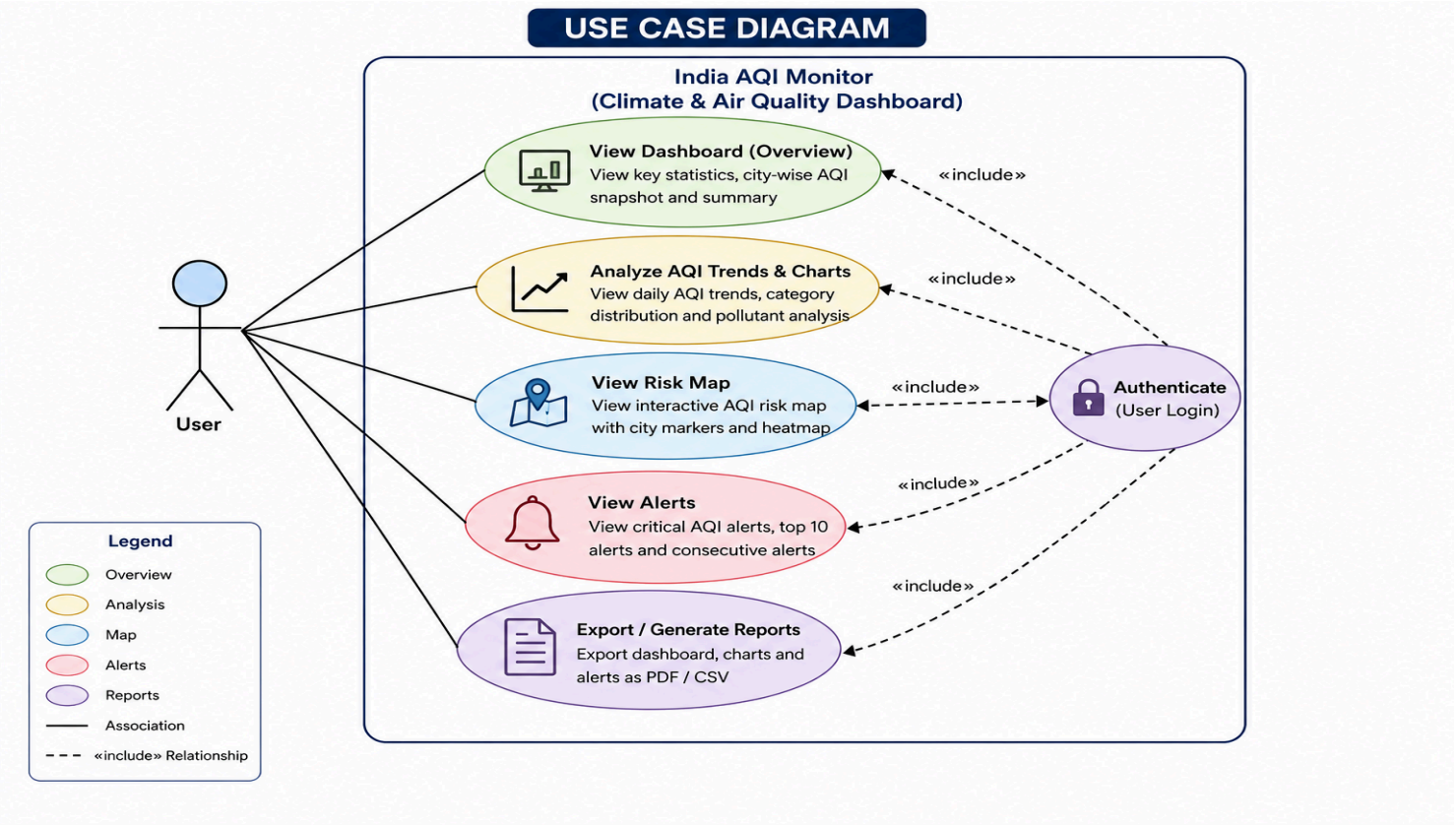
```
                                                    <td        style="padding:3px
6px;font-weight:bold;color:{delta_color};">{row['aqi_delta']:+.1f}</td></tr>
    </table>
  </div>"""

  marker = folium.CircleMarker(
    location=[lat, lon],
    radius=radius,
    color=color, fill=True, fill_color=color, fill_opacity=0.75, weight=2,
    popup=folium.Popup(popup_html, max_width=270),
    tooltip=f"{c} — AQI {row['avg_AQI']:.0f}"
  )
  cat_groups[row['top_category']].add_child(marker)

for grp in cat_groups.values():
  m.add_child(grp)

print("[2] CircleMarkers: radius = avg_AQI / 10, color-coded by severity")
print("[3] Popups: City, Avg AQI, Category, Worst Pollutant, Rush Δ")

# Subtask 4: HeatMap with jitter
rng2 = np.random.default_rng(42)
heat_data = []
```

```
for _, row in df.iterrows():
    c = row['city']
    if c in CITY_COORDS:
        lat = CITY_COORDS[c][0] + rng2.uniform(-0.05, 0.05)
        lon = CITY_COORDS[c][1] + rng2.uniform(-0.05, 0.05)
        heat_data.append([lat, lon, row['AQI'] / 600])

hm_fg = folium.FeatureGroup(name='Heatmap Layer', show=True)
HeatMap(heat_data, radius=18, blur=12, min_opacity=0.3,
        gradient={0.2: 'blue', 0.4: 'lime', 0.6: 'orange', 0.8: 'red', 1.0: 'darkred'}
        ).add_to(hm_fg)
hm_fg.add_to(m)
print(f"[4] HeatMap layer: {len(heat_data)} data points, ±0.05° jitter")
```

FLOWCHART AND UML DIAGRAM



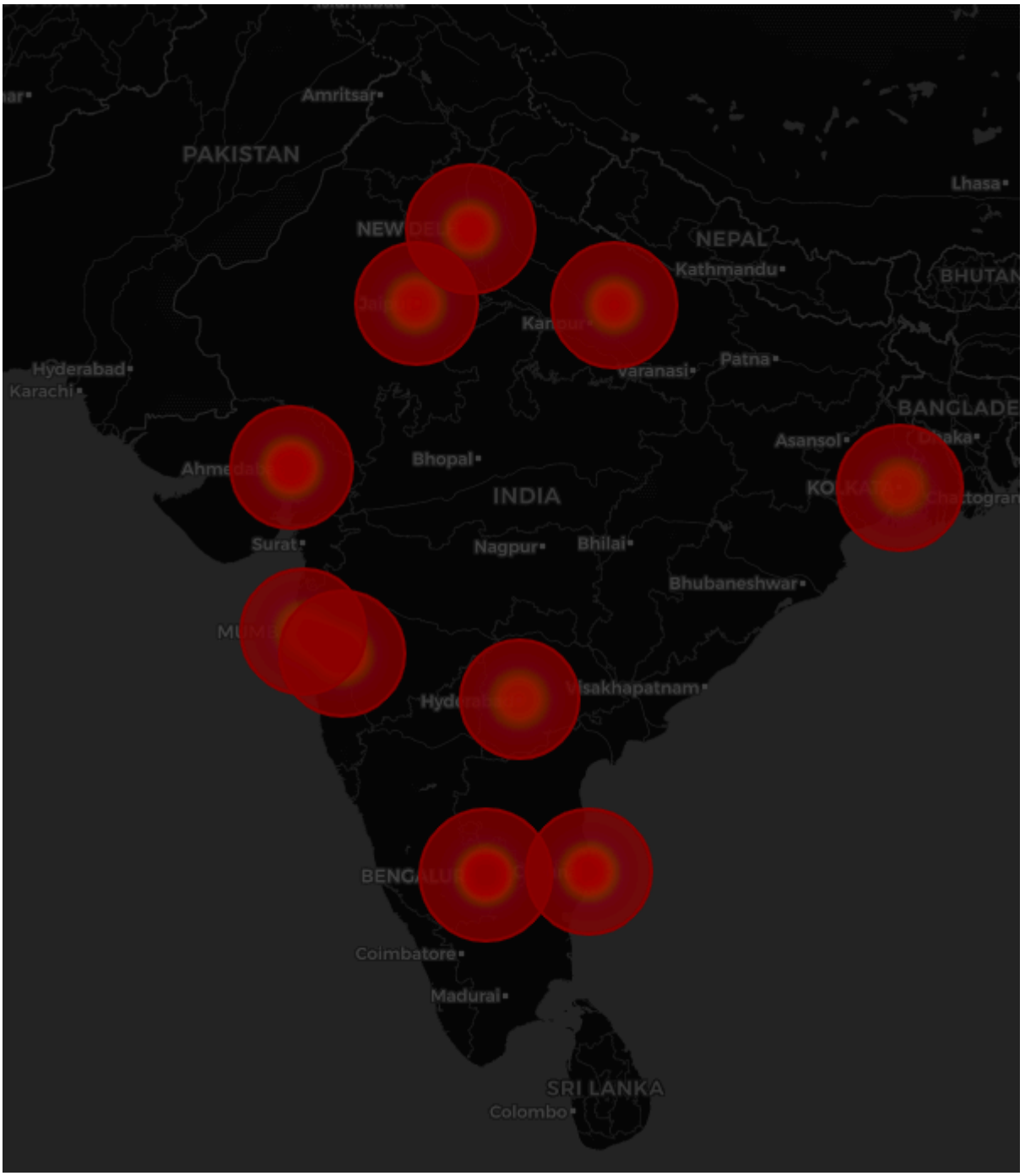
Flow Chart



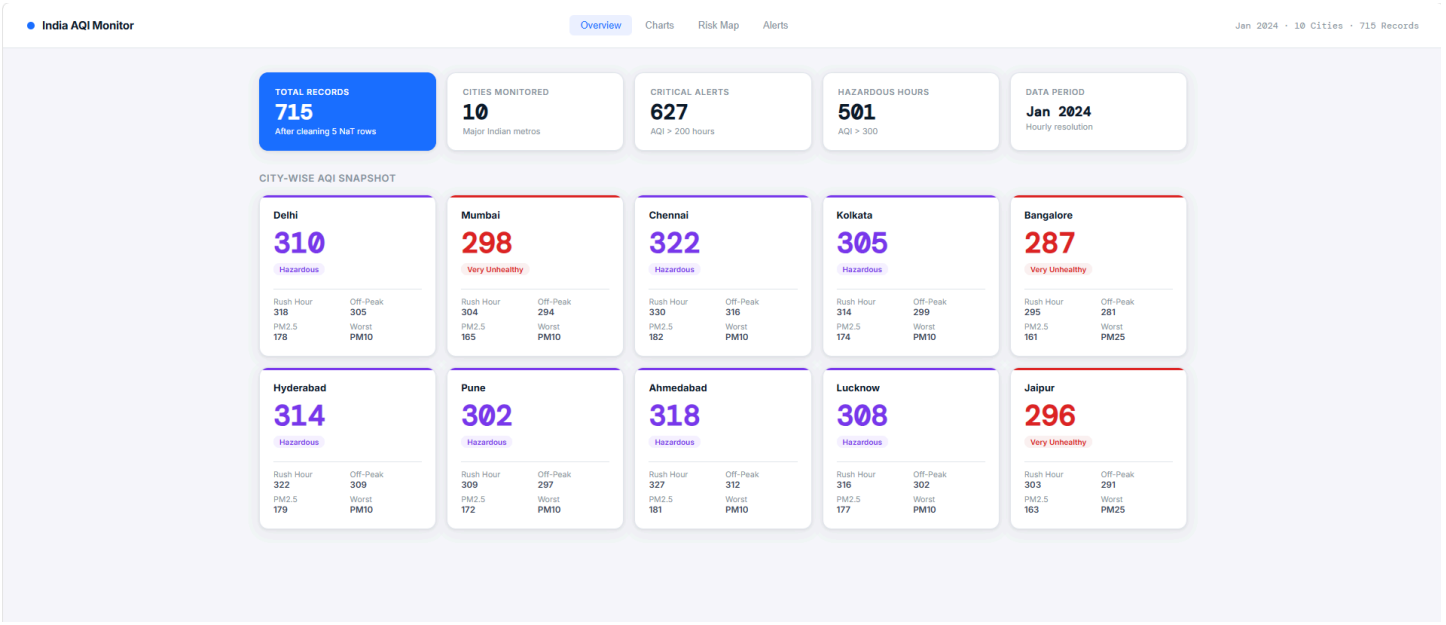
Use Case Diagram

RESULTS AND OUTPUT

Dashboard

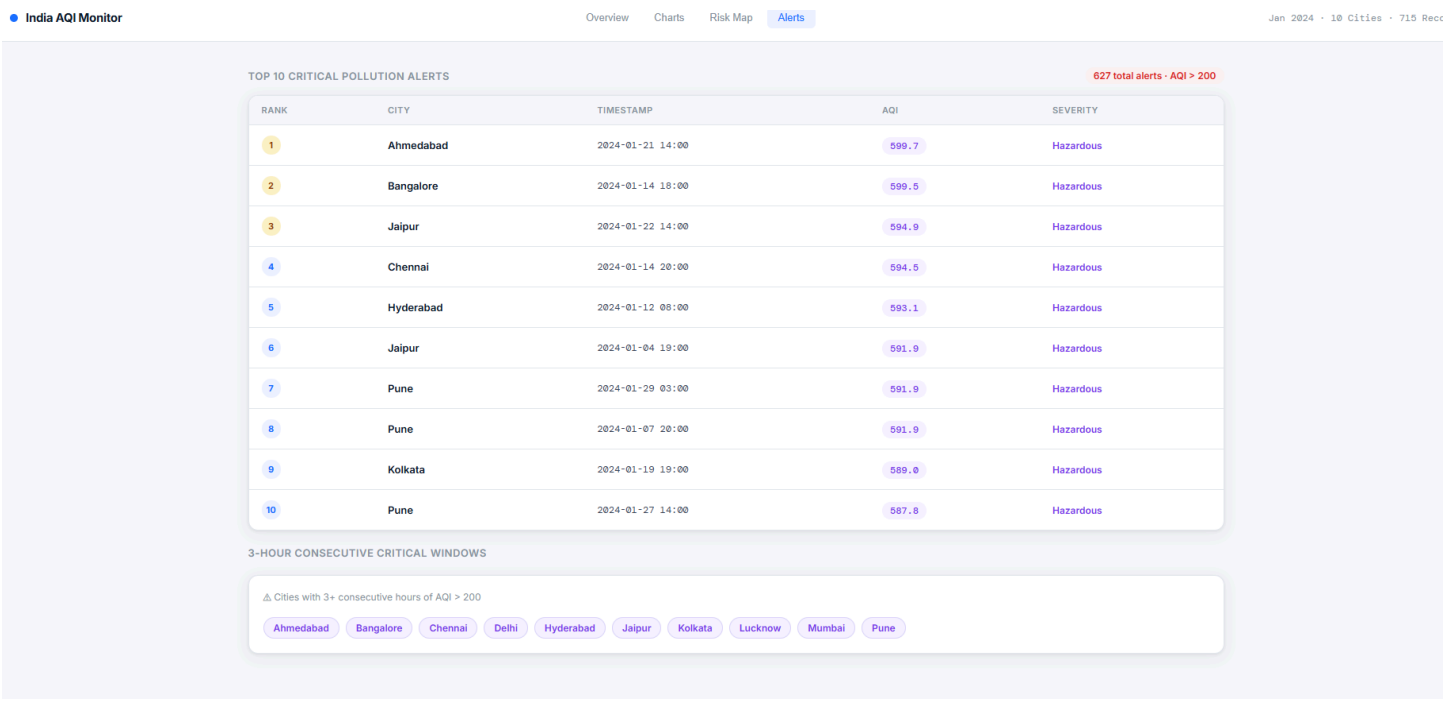

Amritsar
PAKISTAN
Lhasa
NEPAL
Kathmandu
BHUTAN
Hyderabad
Karachi
Varanasi
Patna
BANGLADE
Asansol
Bhopal
INDIA
Surat
Nagpur
Bhilai
Bhubaneshwar
Visakhapatnam
Coimbatore
Madurai
SRI LANKA
Colombo

Risk Map



Web Interface

Charts on Web Interface



High Alert Cities

CONCLUSION

The Climate and Air Quality Monitoring System successfully processes raw pollution sensor data and converts it into meaningful air-quality insights. The project begins with an ETL pipeline that cleans missing, invalid, and inconsistent data values, then calculates AQI using multiple pollutant readings such as PM25, PM10, NO2, SO2, CO, and O3.

The system also classifies AQI into different health-risk categories and identifies rush-hour pollution patterns. Using data structures such as Stack and Priority Queue, the project detects consecutive critical pollution periods and ranks the most severe AQI alerts. This shows how DSA concepts can be applied to a real-world environmental monitoring problem.

The Matplotlib dashboard provides visual analysis through AQI trends, heatmaps, pollutant comparison charts, and city-wise category breakdowns. The Folium-based AQI risk map further improves interpretation by displaying pollution severity geographically across Indian cities.

Overall, this project demonstrates an effective end-to-end solution for air-quality monitoring. It combines data cleaning, AQI computation, alert generation, visualization, and mapping to support better understanding of urban pollution patterns and environmental risk.